University of Ottawa
School of Electrical Engineering and Computer Science

DataGrid Assignment 1 Report

Pointers Functions and Separate Compilation

CSI2372A Programming Concepts with C++

Fall 2026

Student name: Cloris Wong

Student number: 300485139

Submission date: 2026-10-19

Part A Foundations

Provide answers with reasoning for Questions 1 to 4. Predict the results first, then verify them with small compiled tests.

Question 1 Comparing C plus plus and Java

(a) In Java, a DataGrid matrix would be int[][] m = new int[3][5]; and nothing would explicitly free it. What happens in C++ if main never calls grid::destroy? What happens if destroy is called twice on the same matrix, and why does void destroy(int**& m, int rows) prevent that particular problem?

- What happens in C++ if main never calls grid::destroy?
    - Memory leak would happen because the dynamically allocated matrix is not released. Both the array of row pointers and the three row arrays remain allocated. When the local pointer m goes out of scope, its lifetime end. Meanwhile, the dynamically allocated memory is not freed and they would not be accessible.
- What happens if destroy is called twice on the same matrix?
    - Double free may cause the system crack because it is an undefined behavior.
- Why does void destroy(int**& m, int rows) prevent that particular problem?
    - It handles two calls using the same pointer variable. The parameter int**& m allows destroy to set the caller's pointer to nullptr after freeing the matrix. On the second call, the null check makes the function return without accessing or freeing memory again.

(b) After int** t = m; t[0][0] = 99; what is m[0][0]? After vector<int> w = v; w[0] = 99; is v[0] changed? Explain both and what a Java programmer would expect in the second case.

- What is m[0][0]?
    - 99. t pointed to the same address of m. Therefore m[0][0] changed while t[0][0] has assigned a new value.
- Is v[0] changed?
    - No. vector<int> w is an independent copy of the elements.
- Explain both and what a Java programmer would expect in the second case.
    - A Java programmer familiar with array reference assignment might expect both variables to refer to the same array, in which case modifying an element through w would also affect what is accessed through v.

(c) If only grid.cpp changes, which files does make recompile and which does it relink? What if grid.h changes? Why does Java not need the same header and source split?

- If only grid.cpp changes, which files does make recompile and which does it relink?
    - Make recompile grid.cpp into grid.o. It then relinks main.o, stats.o, grid.o, and report.o to produce datagrid.
- What if grid.h changes?
    - Make recompiles both main.cpp and grid.cpp because their object files depend on that header. It then relinks all four object files to produce datagrid.
- Why does Java not need the same header and source split?
    - Java keeps method declarations and implementations together in .java class files. The compiler can obtain type and method information from source files or compiled .class files, so Java does not require a separate header file like C++.

Question 2 Types and expressions

Give each marked value or output and explain it in one line. Treat the statements as a sequence.

```
int a = 7, b = 2, i = 1;
double d = a / b;                 // (1) d = 3.0

double e = double(a) / b;         // (2) e = 3.5
int k = 3.99;                    // (3) k = 3; would Java accept this? No, There is error in Java.
int j = 3 * i++;                  // (4) j = 3 and I = 2
int m = ++i * 2;                  // (5) m = 6 and i = 3
cout << -7 / 2 << " " << -7 % 2;  // (6) output: -3 -1

//(1) When the division is operated, a is still int. integer divided by integer, the result would be
a integer. Then this result is assigned to a double variable.

//(2) Before the division operated, a has turned to a double variable. Therefore the result is a
double. Then this result is assigned to a double variable.

//(3) k is an integer. Therefore the decimal number is ignored.

//(4) 3 * I, this result assigns to m. After this operation, i increased by 1.

//(5) i increased by 1 before the calculation. The i after increment multiply 2. Then this result
assigns to m

//(6) Integer division truncates toward zero, so -7 / 2 is -3. The remainder is -1 because -7 = (-3)
* 2 + (-1)
```

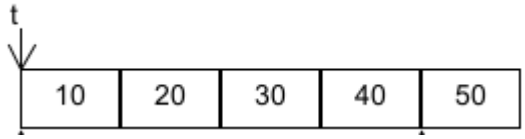
Part A Pointers and parameter passing

Question 3 Pointer arithmetic

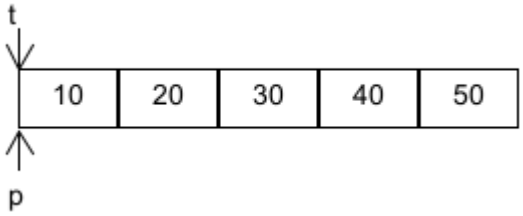
```
int t[] = {10, 20, 30, 40, 50};
int* p = t;
int* q = t + 4;
*(p + 1) += 5;
p += 2;
*p = *q - *(p - 2);
q--;
cout << *p << " " << q - p << " " << t[1] << " "
     << *(q - 1) << "\n";
```

(a) Draw the array and the positions of p and q after each statement, and give the output.

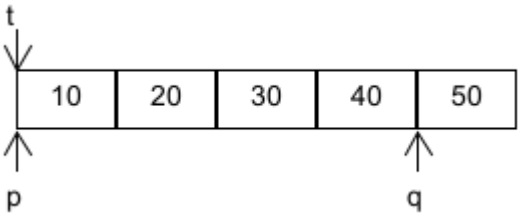
-



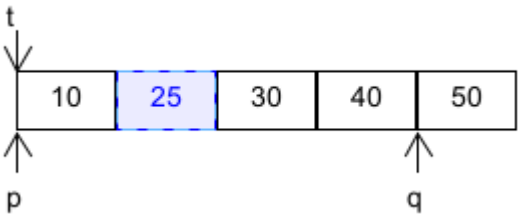
-



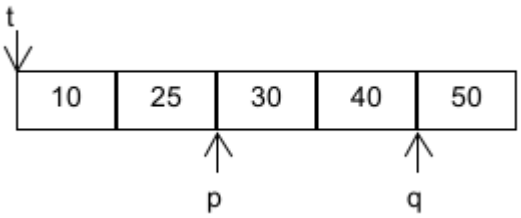
-



-

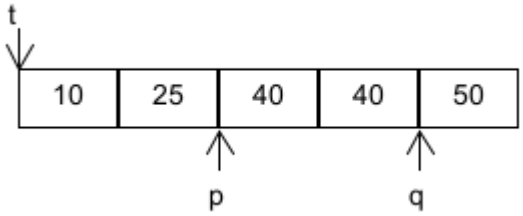


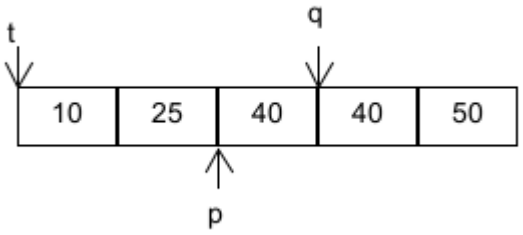
-



-

- *p = *q - *(p - 2); /*p = 30, *q = 50 and *(p - 2) = 10





-
- Output: 40 1 25 40

(b) Is int* r = t + 5; legal? Is *r legal? Explain the difference.
- Is int* r = t + 5; legal?
  - Legal. This pointer is just pointing to a position past the last element of an array. It is not pointing to any elements in the array.
  - Illegal. The r does not point to an existing array element, so the dereferencing *r accessing this position cause an undefined behavior.

Question 4 Overloading and parameter passing

report.h declares print(const std::string&, int) and print(const std::string&, double). For each call, state whether it compiles, which overload is chosen, and what it prints.

```
report::print("x", 'A'); //Compile success, chosen overload: int, print: "  x         : 65"
report::print("x", 2.5f); //Compile success, chosen overload: double, print: "  x         : 2.50"
report::print("x", true); //Compile success, chosen overload: int, print: "  x         : 1"
report::print("x", 7L); //Compile fail, reason: Both long->int and long->double are standard
conversion, there is no priority to choose any of them(Ambiguous Overload)
```

Explain why stats has two minMax versions, one taking int* lo and int* hi, and one taking int& lo and int& hi. Write one call to each version. Explain why the vector parameter is const std::vector<int>& rather than std::vector<int>.

- why stats has two minMax versions, one taking int* lo and int* hi, and one taking int& lo and int& hi.
  - The two versions allow the caller to receive the minimum and maximum values using either pointers or references. Both versions can modify the caller's lo and hi variables. The pointer version uses addresses and dereferencing, while the reference version provides direct access to the original variables.
  - *Pointer version:*
    int nums[] = {3, 1, 5};
    int lo, hi;
    stats::minMax(nums, 3, &lo, &hi);
  - *Reference version:*
    std::vector<int> v = {3, 1, 5};
    int lo, hi;
    stats::minMax(v, lo, hi);
- Why const std::vector<int>&?

- The vector is passed by const reference to avoid copying the entire vector, improving efficiency. The const keyword also prevents the function from modifying the original vector. Passing by value would create an unnecessary copy.

Part C Memory and linkage

Question 5 The four pitfalls of dynamic memory

```
int** m = grid::create(3, 4);
int* s = grid::rowSums(m, 3, 4);
s = grid::rowSums(m, 3, 4);        // (1)
int** t = grid::transpose(m, 3, 4);
delete t;                          // (2)
int** alias = m;
grid::destroy(m, 3);
cout << alias[0][0];               // (3)
grid::destroy(alias, 3);           // (4)
delete[] s;
```

(a) Identify the bugs at (1) to (4): memory leak, dangling pointer, double free or wrong delete. Explain precisely which memory is involved. Line (2) has two problems; give both.

- (1): Memory leak. The second call to rowSums allocates another array and overwrites s. The first row-sum array is never freed, and its address is lost.
- (2a): Wrong delete. The row-pointer array of t was allocated with new[], so using delete t instead of delete[] t causes undefined behavior.
- (2b): Memory leak. The transpose has four separately allocated row arrays, each containing three integers. These rows must be freed individually before freeing the row-pointer array; delete t does not do that.
- (3): Dangling pointer access. alias still holds the address of the original matrix after grid::destroy(m, 3) frees it. Accessing alias[0][0] uses freed memory, causing undefined behavior.
- (4): Double. grid::destroy(alias, 3) attempts to destroy the already-freed original matrix again. Reading its freed row-pointer array is already invalid, and attempting to free the allocations again causes a double-free error.

(b) destroy sets m to nullptr. Why does that not protect alias?

- Setting m to nullptr does not protect alias because they are separate pointer variables. The assignment int** alias = m; copies the address, not a reference to the pointer variable m. After the matrix is freed, only m is reset; alias retains the invalid address and becomes a dangling pointer.

(c) Rewrite the snippet without memory errors.

- int** m = grid::create(3, 4);

```
int* s = grid::rowSums(m, 3, 4);
delete[] s;                // Free the first array before replacing it.
s = grid::rowSums(m, 3, 4);

int** t = grid::transpose(m, 3, 4);
grid::destroy(t, 4);       // The transpose has 4 rows.

int** alias = m;
cout << alias[0][0];       // Access the matrix before freeing it.

grid::destroy(m, 3);       // Free the original matrix exactly once.
alias = nullptr;           // Clear the remaining dangling pointer.

delete[] s;                // Free the second row-sum array.
s = nullptr;
```

Question 6 Compile error link error or clean build

Each scenario independently changes the finished project. Classify the result, identify the file for a compile error, and explain why.

(a) In report.cpp, the definition becomes static int reportsPrinted = 0;
- Compile error in report.cpp: static conflicts with the earlier extern declaration in report.h

(b) In stats.cpp, the definition becomes void sortValues(int* a, int n, bool ascending = true) { ... }.
- Compile error in stats.cpp: The default argument is repeated; it is already specified in stats.h

(c) In main.cpp, extern int reportsPrinted; is replaced by int reportsPrinted = 0;.
- Link error: reportsPrinted is defined in both main.cpp and report.cpp.

(d) In report.h, inline is removed from inRange.
- Link error: Without inline, including report.h in multiple source files creates multiple definitions of inRange

(e) In main.cpp, int v[3] = {1, 2, 3}; cout << stats::mean(v, 3); is added to main.
- Clean build: v converts to a pointer to its first element, matching mean's parameter; it prints 2

Part C Functions and report discussion

Question 7 Recursion and static locals

(a) For int a[] = {5, 1, 7, 3}; trace countAbove(a, 4, 3). List every call with its arguments using a + k, each returned value, and the maximum number of simultaneously active calls.
- countAbove(a + 0, 4, 3), return: 2
- countAbove(a + 1, 3, 3), return: 1
- countAbove(a + 2, 2, 3), return: 1
- countAbove(a + 3, 1, 3), return: 0
- countAbove(a + 4, 0, 3), return: 0
- maximum number of simultaneously Active calls: 5

(b) If static is removed from the counter in report::header, what does b1_basic.txt print in its headers? Is the final report count still correct? Explain why.
- If static is removed from the counter in report::header, what does b1_basic.txt print in its headers?
  - Every header prints Report #1. Without static, the local counter is initialized to 0 on every call, then incremented to 1.
- Is the final report count still correct? Explain why.
  - Yes, the final count remains correct: 8 report(s) printed. The separate global variable reportsPrinted still retains its value and is incremented on every call.

Question 8 Function pointers and generic pointers

(a) Declare HANDLERS without typedef Handler. Declare a pointer op that can hold stats::square, and show a call through it.
- ```
  static void (*HANDLERS[])(int**, int, int, std::istringstream&) = {
     cmdPrint, cmdRow, cmdCol, cmdAbove, cmdSums,
     cmdTranspose, cmdSort, cmdApply, cmdMax
  };

  int (*op)(int)  = stats::square;
  int result = op(5);  // Returns 25.
  ```

(b) Why does a qsort comparator receive const void* parameters? What does static_cast<const int*>(x) do? Why is return a - b; incorrect for 2000000000 and -2000000000?
- Why does a qsort comparator receive const void* parameters?
  - qsort supports different element types, so it passes generic pointers. const prevents the comparator from modifying elements through those pointers
- What does static_cast<const int*>(x) do?
  - It converts the generic pointer into a pointer to const int, allowing the comparator to dereference it as an integer
- Why is return a - b; incorrect for 2000000000 and -2000000000?
  - 2000000000 - (-2000000000) equals 4000000000, which exceeds a 32-bit signed int and causes undefined behavior

Program design

grid::create allocates the row-pointer array and each row using new[]. main owns this matrix and calls grid::destroy, which frees each row, then the pointer array, and sets the caller's pointer to nullptr. grid::rowSums allocates a result array that cmdSums frees with delete[]. grid::transpose allocates a new matrix that cmdTranspose frees using grid::destroy(t, cols). Pointers returned by stats::findMax refer to existing elements and must not be deleted.
  ```

The dispatch table uses two parallel arrays: NAMES stores command names, and HANDLERS stores their function pointers in the same order. lookup finds the matching name and returns its handler, which main calls. If no match exists, it returns nullptr, and main prints an unknown-command error.

Testing

custom1_mutations.txt tests sorting, transformations, first-occurrence maximum selection, strict threshold comparison, column means, row sums, and transpose without changing the stored matrix. Its output matches the expected results and contains 11 reports.

custom2_errors.txt tests seven invalid commands followed by valid commands. Each error produces the correct message without increasing the report count. Execution continues correctly, and commands after quit are ignored. The output contains four reports.

Both tests exit with code 0; their outputs are submitted as custom1_mutations.actual.out and custom2_errors.actual.out. The submitted Valgrind logs for b1, b3, and b5 each report zero memory errors and zero bytes remaining allocated at exit.

Challenges and lessons learned

This was my first time reading a complete C++ project. I found it challenging because I was unfamiliar with the different file types and syntax. Although the lectures introduced these concepts, it took me time to recognize them in actual code, especially namespaces and function declarations. For example, I initially did not understand std:: or where the names following it came from. I learned that std is the namespace used by the C++ standard library, and :: identifies a name within that namespace. After studying the examples in main.cpp, I became more comfortable understanding the remaining functions and implementing their logic.

I tried writing the logic myself and debugging problems before asking AI for explanations or help improving my code. AI assistance made my work more efficient and helped me understand unfamiliar concepts faster. However, I still needed to review the suggestions and understand how they worked. Otherwise, it would give me a lot of unnecessary files and changes.

Building the program was also different from my previous experience with Python and Java. I initially misunderstood the purpose of the Makefile, thinking it might be program output or an AI-generated file. I learned that it is a build configuration file provided with the project. The make tool reads its rules to determine which source files need recompiling and how to link the object files into an executable. A Makefile is not built into C++ and is not required for every C++ project.

I also learned how stats_test.cpp collects multiple tests into one program. After compilation, stats_test.exe runs these tests together and reports their results. Previously, I usually ran test cases individually in the terminal, so this approach made checking my work more efficient.

Finally, I learned about vcvars64.bat, a Visual Studio script that configures environment variables so the terminal can find the MSVC compiler, headers, and libraries. It does not compile the code itself; it prepares the environment for the cl compiler. This helped me distinguish between source files, build tools, compiler setup, and executable programs.